# AI FOR DEVELOPERS

Beginner Handbook

## 1. THE AI LANDSCAPE (The Big Picture)

AI is a broad field. As developers, it's important to understand how all the key concepts are connected.

★ **Artificial Intelligence (AI):**
The ability of machines to simulate human intelligence and perform tasks that normally require human intelligence.

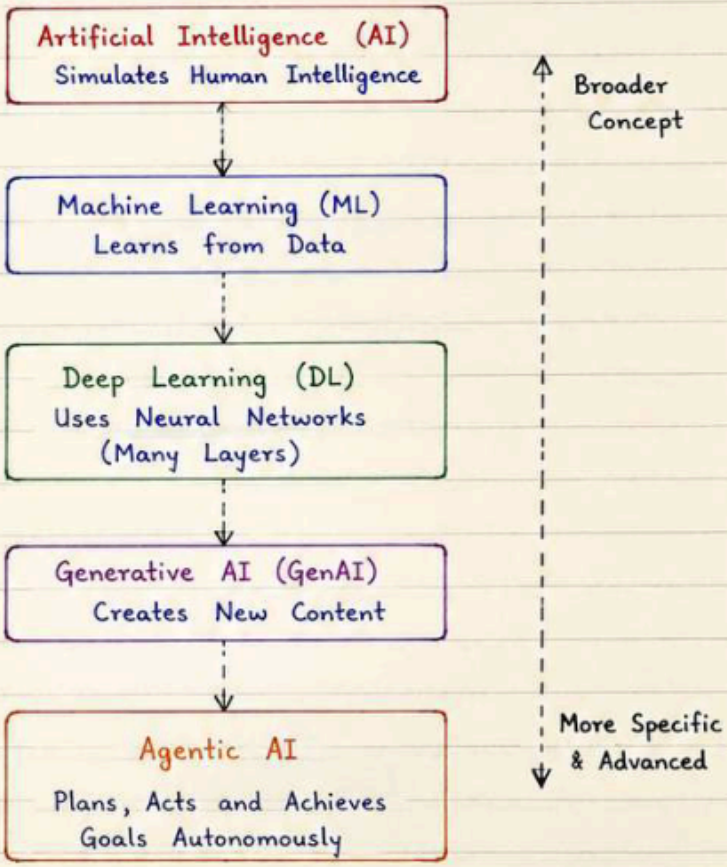
★ **Machine Learning (ML):**
A subset of AI where machines learn patterns from data and improve automatically without being explicitly programmed.

★ **Deep Learning (DL):**
A subset of ML that uses neural networks with many layers to learn complex patterns from large amounts of data.

★ **Generative AI (GenAI):**
A type of AI that can generate new content (text, image, code, audio, video, etc.) based on what it has learned.

★ **Agentic AI:**
AI systems that can think, plan, take actions using tools, and achieve goals with minimal human intervention.

### How Everything Fits Together




### Quick Comparison

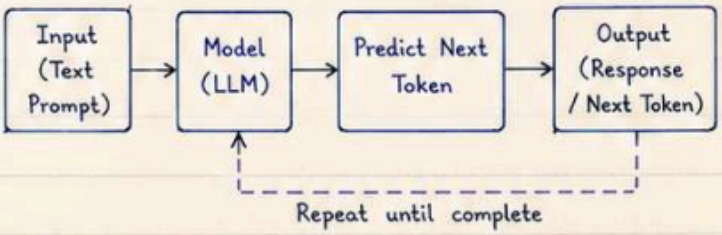
| Concept | Focus | Example | Used For |
|---|---|---|---|
| AI | Simulate human intelligence | Chatbots, Self-driving cars | General problem solving |
| ML | Learn from data | Spam detection, recommendations | Pattern learning & prediction |
| DL | Learn using deep neural networks | Image recognition, speech recognition | Complex data (images, audio, etc.) |
| GenAI | Generate new content | ChatGPT, DALL·E, GitHub Copilot | Content creation, automation |
| Agentic AI | Act autonomously to achieve goals | AutoGPT, AI Agents with tools | Task automation, decision making |

**Key Takeaway** → All these are connected. As you go down, systems become more powerful, capable and independent.

# 2. HOW LLMS WORK

LLMs (Large Language Models) are the brain behind most AI tools like ChatGPT, Claude, Gemini, etc. They understand text and generate human-like responses.

## HOW LLM GENERATES THE NEXT TOKEN




## ★ WHAT IS AN LLM?

An LLM is a model trained on huge amounts of text data. It learns patterns in the text and uses those patterns to predict the next word (or token) in a sequence.

**KEY IDEA**

LLM does not "know" facts.
It predicts the next token based on patterns it has seen during training.

## ★ MAIN COMPONENTS

① Training Data : Model learns from billions of words from internet, books, articles, code, etc.

② Tokens : Text is broken into small pieces called tokens.

③ Model (Transformer): Uses transformer architecture to find patterns and relationships between tokens.

④ Inference : Given a prompt, model predicts the next token one by one to generate a response.

## TRAINING PROCESS (IN SHORT)

1. Collect Huge Text Data
2. Learn Patterns (Predict Next Word)
3. Large Model (Transformer Architecture)
4. Model is Ready for Inference (Use)

## EXAMPLE (NEXT TOKEN PREDICTION)

Prompt: The sky is
Model predicts next tokens:

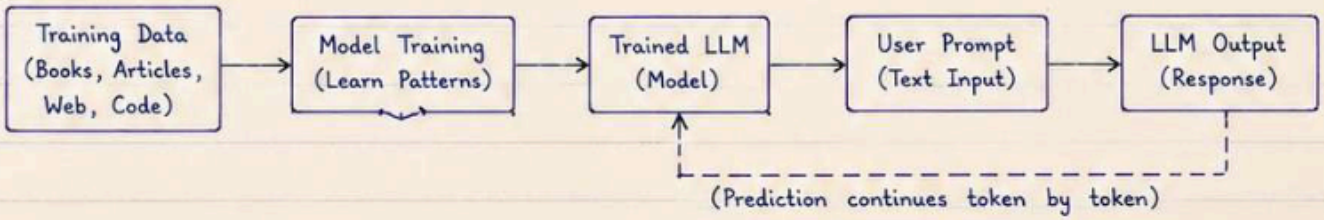
Step 1 → blue
Step 2 → and
Step 3 → clear
Step 4 → with
Step 5 → clouds

At each step, the model chooses the most likely next token and continues the generation.

## IMPORTANT POINTS

- LLMs generate text, not facts.
- They memorize patterns, not specific information.
- The more data and better training, the better performance.
- LLMs are the base for tools like ChatGPT, Copilot, etc.

## SIMPLE VIEW OF LLM WORKFLOW



# 3. PROMPT ENGINEERING

Prompt engineering is the art of asking the right questions so that LLMs give accurate, useful and consistent responses.

**KEY IDEA**
Better Prompt
=
Better Response

## TYPES OF PROMPTS

① **Zero-shot Prompting :**
Ask directly without any example.
(Good for simple questions)

② **One-shot Prompting :**
Give one example, then ask.
(Better for clarity)

③ **Few-shot Prompting :**
Give multiple examples, then ask.
(Best for accuracy)

④ **Chain-of-Thought (CoT) :**
Ask the model to think step-by-step before answering.

⑤ **Role Prompting :**
Assign a role to the model
(e.g. You are a Python Expert)

⑥ **Context Prompting :**
Provide background/context so model understands the situation.

## PROMPT TEMPLATE (USE THIS FORMAT)

| | |
|---|---|
| Role : | Define who the model is. |
| Task : | What you want the model to do. |
| Context : | Provide background information. |
| Rules : | Give important rules or constraints. |
| Format : | How the answer should be given. |
| Input : | Actual data or question. |
| Output : | Expected response. |

## BAD PROMPT vs GOOD PROMPT

| BAD PROMPT | GOOD PROMPT |
|---|---|
| Explain Python lists. | You are a Python expert. Explain Python lists with examples. Include methods, operations and a short code example. |
| ↓ | ↓ |
| (Too vague) | (Clear & Specific) |
| Response → General, short, not useful. | Response → Detailed, structured & useful. |

## CHAIN OF THOUGHT (CoT) EXAMPLE

**Prompt :**
Q. If a train travels 60 km in 1 hour, how far will it travel in 4.5 hours? Think step-by-step.

**Step-by-Step Reasoning :**
Step 1 → Speed = 60 km / 1 hour = 60 kmph
Step 2 → Time = 4.5 hours
Step 3 → Distance = Speed × Time
Step 4 → Distance = 60 × 4.5 = 270 km
**Answer :** 270 km

## TIPS FOR BETTER PROMPTS

① Be clear and specific.
② Give enough context.
③ Break complex tasks into steps.
④ Give examples (if needed).
⑤ Tell the model how to format the answer.
⑥ Ask the model to think step-by-step.
⑦ Iterate and improve the prompt.

Small change in the prompt can give big change in the output.

**Remember :** : LLMs are powerful, but your prompt is the real power.

# 4. RAG (RETRIEVAL-AUGMENTED GENERATION)

RAG helps LLMs use your own data to give accurate and up-to-date answers.

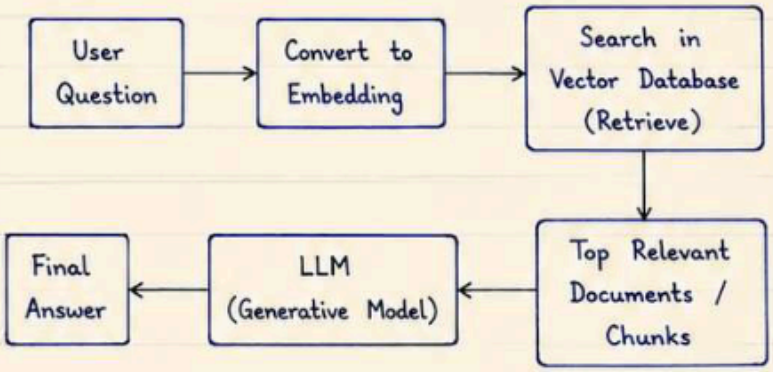
## WHY RAG?

- LLMs have knowledge cutoff.
- They can hallucinate (make things up).
- RAG provides real, domain-specific information.

## WHAT IS RAG?

RAG combines retrieval of relevant information from a knowledge source with the generative power of an LLM to produce accurate answers.

## HOW RAG WORKS (FLOW)




In short :

Retrieve relevant info → Give to LLM → LLM generates better answer.

## RAG PROCESS EXPLAINED

① Ingestion : Load documents (pdf, txt, docs, web pages, etc.) Split into smaller chunks.

② Embedding : Convert each chunk into vector (numbers). Store in Vector Database.

③ Retrieval : User asks question → convert to vector → find similar chunks.

④ Augmentation : Send retrieved chunks + question to LLM.

⑤ Generation : LLM uses that context to generate accurate answer.

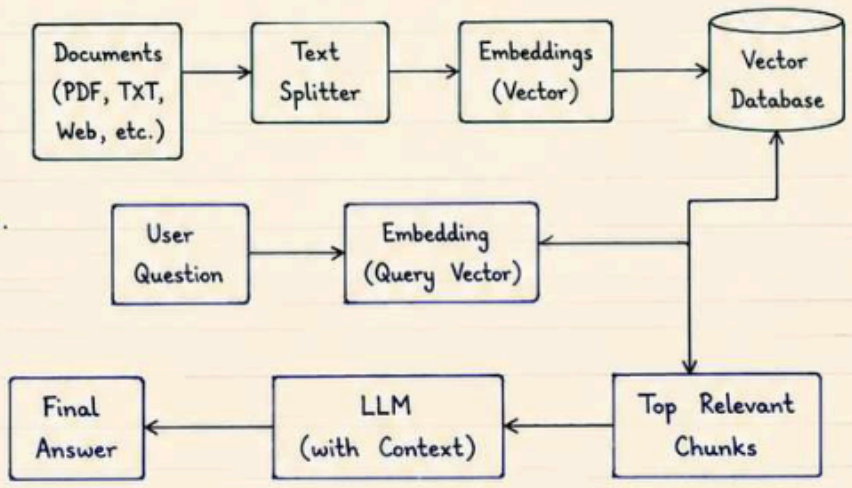
## RAG vs NORMAL LLM

| Normal LLM | RAG |
|---|---|
| • Uses only training data. | • Uses your own data. |
| • May be outdated. | • More accurate. |
| • Can hallucinate. | • Less hallucination. |
| • No access to your own documents. | • Always up-to-date with your data. |

## WHERE IS RAG USED?

- Chat with PDFs / documents
- Company internal knowledge bots
- Customer support bots
- Research assistants
- Legal, finance, healthcare, education, etc.

## SIMPLE RAG ARCHITECTURE




TIP

The quality of RAG depends on your data quality, chunk size and embeddings.

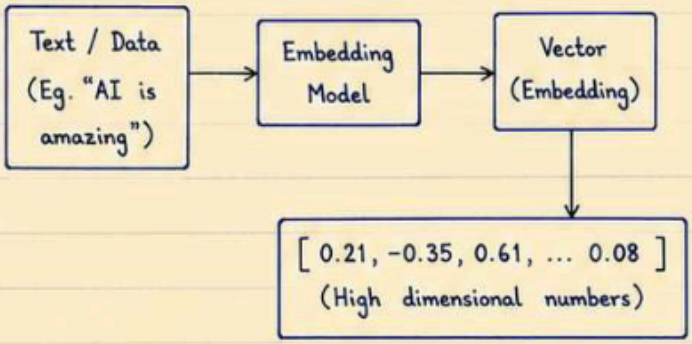
Remember : RAG = Real Data + Retrieval + LLM = Better Answers

# 5. EMBEDDINGS & VECTOR DATABASES

## WHAT ARE EMBEDDINGS?

Embeddings are numerical (vector) representations of text, images or data in a high dimensional space where similar items are placed closer together.

## HOW EMBEDDINGS WORK (OVERVIEW)




## WHY EMBEDDINGS?

- Machines understand numbers, not words.
- Captures meaning / context of text.
- Helps in similarity search.
- Useful for RAG, search, recommendations, etc.

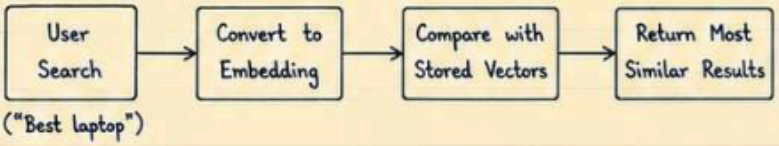



**KEY IDEA :**

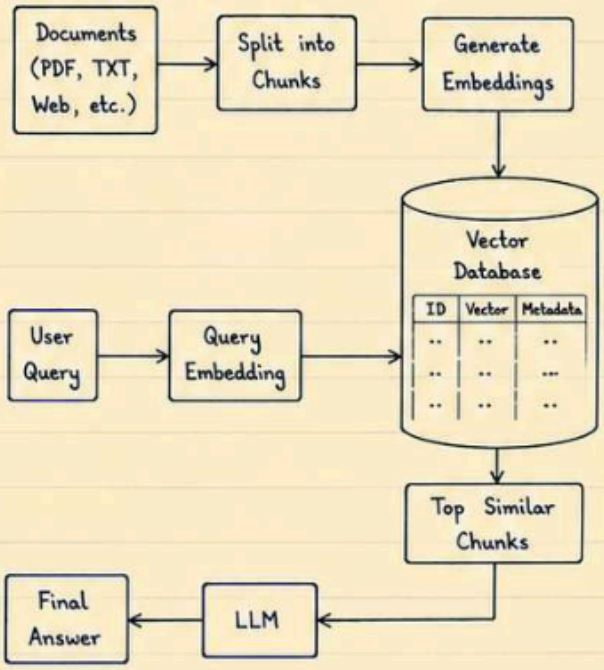
Similar meaning → Vectors are closer
Different meaning → Vectors are farther

Distance is calculated using metrics like Cosine Similarity.

## SIMILARITY SEARCH (USING EMBEDDINGS)




## HOW VECTOR DATABASE WORKS




## WHAT IS A VECTOR DATABASE?

A vector database is a special type of database that stores embeddinas and allows fast similarity search on high dimensional vectors.

Examples :

- Pinecone
- Chroma
- Weaviate
- FAISS
- Milvus

**Vector DB stores :**

- Vector (embedding)
- Original Text / Data
- Metadata (id, source, page no., etc.)

It is optimized for similarity search, not exact match.

## EMBEDDINGS + VECTOR DB = POWERFUL COMBINATION

- Semantic Search (meaning based search)
- RAG (Retrieval Augmented Generation)
- Chat with your own data
- Recommendations, clustering, classification, etc.

**TIP :**

Good embeddings model + Good chunking + Good vector DB = High quality results.

★ Embeddings turn text into numbers, Vector DB helps find the most similar meaning.

@Aman_Views

# 6. AI AGENTS & AGENTIC AI

## WHAT IS AN AI AGENT?

An AI Agent is a system that can perceive its environment, think, take actions and achieve goals on behalf of the user.
It can use tools, remember things and improve its decisions.

## TYPES (LEVELS)

① AI Assistant :
Answers questions, helps with tasks.
(e.g. ChatGPT)

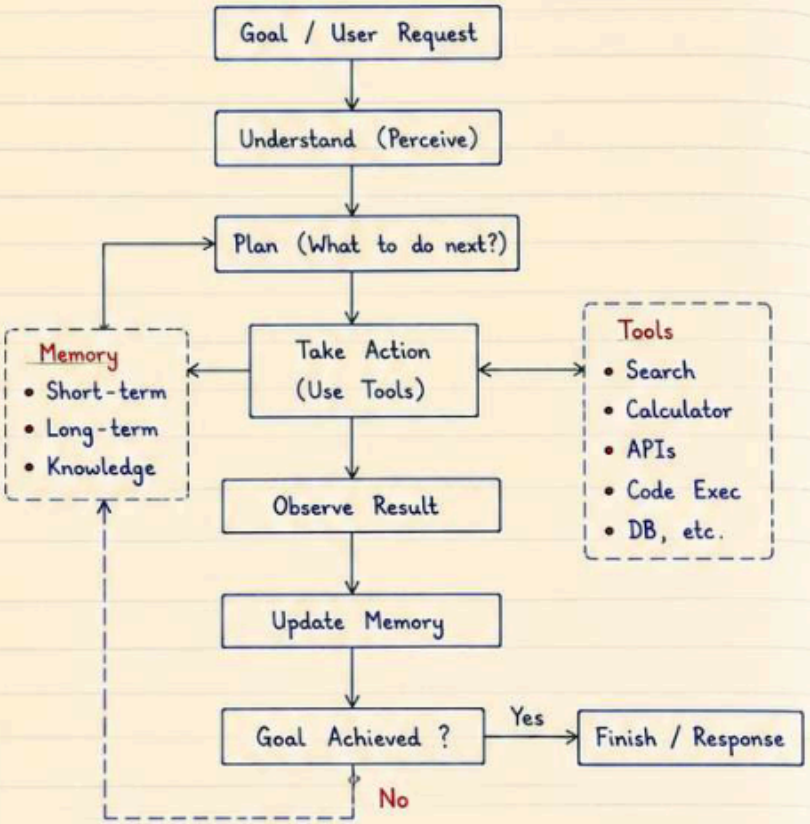
② AI Agent :
Can use tools, access data, perform tasks.

③ Agentic AI :
Autonomous, plans, learns, and makes decisions to achieve complex goals.

**KEY DIFFERENCE**

AI Assistant → Responds
AI Agent → Acts
Agentic AI → Thinks, Plans, Acts, Improves

## HOW AN AI AGENT WORKS (FLOW)




## CORE CAPABILITIES OF AN AI AGENT

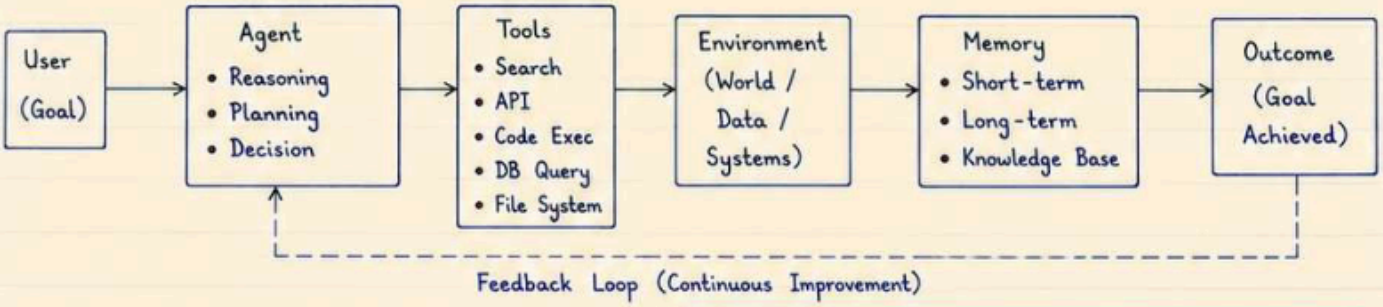
① Autonomy : Can work with minimal human help.
② Reasoning : Understands and reasons about tasks.
③ Planning : Breaks down big goals into steps.
④ Tool Use : Uses external tools / APIs.
⑤ Memory : Remembers context and past interactions.
⑥ Learning : Improves from feedback and experience.

## EXAMPLE : AI AGENT (TRAVEL PLANNER)

Goal : Plan a trip from Delhi to Goa next weekend

| Step | What Agent Does |
|---|---|
| 1. Understand | Understands location, date, budget |
| 2. Plan | Breaks down: flights, stay, places, etc. |
| 3. Action | Search flights, hotels, weather using tools |
| 4. Observe | Collects results from tools |
| 5. Decide | Chooses best options |
| 6. Respond | Gives final travel plan to user |

## AGENTIC AI ARCHITECTURE (HIGH LEVEL)




**PRO TIP**

Start small : Use AI agents for specific tasks.
Combine LLM + Tools + Memory = Powerful Agent.

**IMPORTANT POINT**

★ Agent works in a loop until goal is achieved.
★ Tools + Memory make the agent powerful.
★ Agentic AI can handle complex, multi-step tasks.

@Aman_Views

# 7. MCP (Model Context Protocol)

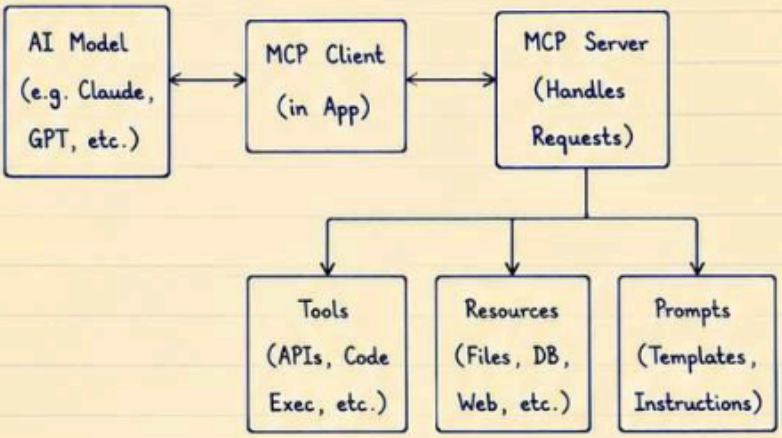
## WHAT IS MCP?

MCP is an open protocol that allows AI models to connect with external tools, data sources and systems in a standardized way.

## WHY MCP?

- Makes AI models more useful and powerful.
- Easy integration with tools and data.
- Secure and controlled access.
- Works across different platforms and models.

## HOW MCP WORKS (HIGH LEVEL)




## MCP COMPONENTS

① MCP Host / Client : Application that uses MCP to connect with server.

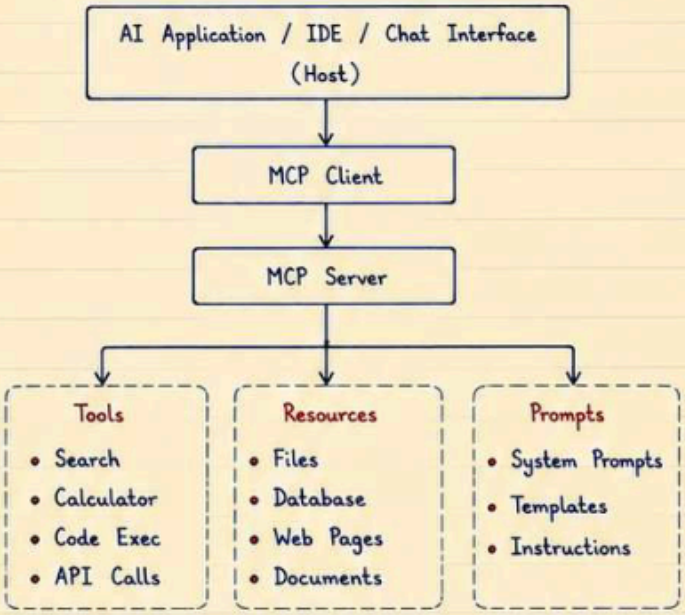
② MCP Server : Exposes tools, resources and prompts through MCP.

③ Tools : Functions that AI can call (e.g. Search, Calculator, Code Runner).

④ Resources : Data sources like files, DB, APIs, documents.

⑤ Prompts : Reusable prompt templates for consistent interactions.

## MCP ARCHITECTURE (DIAGRAM)




## MCP vs Traditional Integration

| Traditional | MCP |
|---|---|
| Custom Integration | Standardized Protocol |
| Hard to Scale | Easy to Add New Tools |
| Different for each App | Works Across Apps |
| More Code & Logic | Simple & Reusable |

## MCP BENEFITS

① Standard way to connect AI with external world.

② Reduces custom code and complexity.

③ Easy to extend with new tools and data sources.

④ Improves security and control.

⑤ Future proof and model agnostic.

## REAL WORLD EXAMPLE

You are using an AI coding assistant in your IDE.

Through MCP it can :

- Read your project files
- Search documentation
- Run tests
- Deploy code
- All without you leaving the IDE!

Example Flow :

You ask a question
↓
MCP Client sends request
↓
MCP Server uses tool/resource
↓
Result comes back to AI
↓
AI gives answer to you

### MCP IN SHORT

MCP is like a universal connector between AI models and the tools / data / systems they need to get real work done.

**Remember :** MCP = Standard Connection + Tools + Resources + Prompts = More Powerful AI

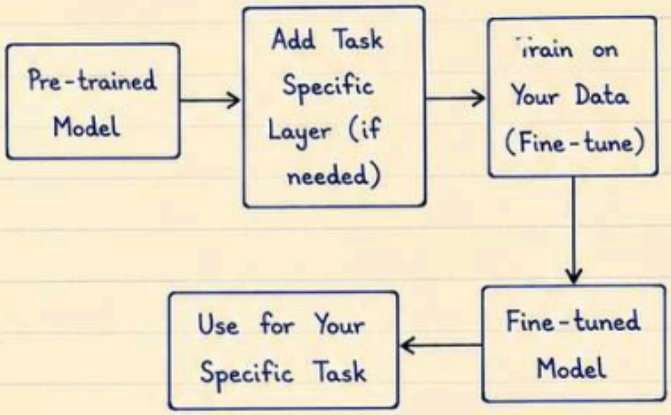
@Aman_Views

# 8. FINE-TUNING & ADAPTATION

## WHAT IS FINE-TUNING?

Fine-tuning means training a pre-trained model on specific data so that it performs better on a particular domain or task.

## HOW FINE-TUNING WORKS (FLOW)




## WHY FINE-TUNE?

- Pre-trained models are general.
- Fine-tuning makes them specific and accurate.
- Helps in better results for your use case.
- Requires less data compared to training from scratch.

## FINE-TUNING METHODS

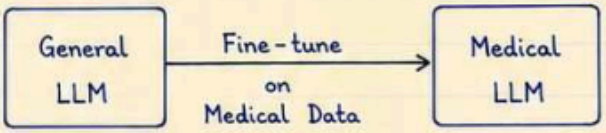
① Full Fine-tuning :
Update all parameters of the model.
(High accuracy, more training time)

② Feature-based Fine-tuning :
Freeze some layers, train only last few layers.
(Less training time, good results)

③ Prompt-based Fine-tuning (PEFT) :
Use techniques like LoRA, Adapter, Prefix-tuning.
(Very efficient, less memory)

## EXAMPLE

You have a general LLM.
You fine-tune it on medical data.
Now it can answer medical related questions much better.



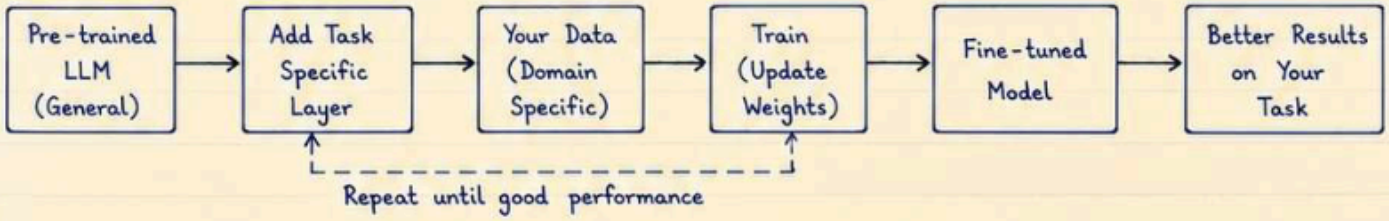

## COMMON USE CASES

- Customer support chatbot
- Legal document analysis
- Medical assistant
- Code generation in specific languages
- Domain specific Q&A

## FULL FINE-TUNING vs PEFT (e.g. LoRA)

| Feature | Full Fine-tuning | PEFT / LoRA |
|---|---|---|
| Parameters Updated | All | Few (Adapters) |
| Data Required | More | Less |
| Training Time | More | Less |
| Memory Usage | High | Low |
| Performance | High | Good (Efficient) |
| Cost | High | Low |

## SIMPLE DIAGRAM - FINE-TUNING (EXAMPLE)




**TIP**
Start with PEFT (like LoRA).
If more performance needed, then go for Full Fine-tuning.

**IMPORTANT POINTS**
★ Good data + Good model + Good fine-tuning = Best results.
★ Quality of data is more important than quantity.
★ Always evaluate model on real world data.

**Remember :** Pre-trained model gives power, Fine-tuning gives accuracy for your use case.

@Aman_Views

# 9. AI DEVELOPER STACK

## WHAT IS AI DEVELOPER STACK?

AI Developer Stack is a combination of models, tools, frameworks and services that help developers build AI powered applications.

## WHY STACK MATTERS?

- Right model for the right task.
- Better performance and cost.
- Easy integration and scalability.
- Faster development and deployment.

## AI MODELS (FOUNDATION MODELS)

| Provider | Model Family |
|---|---|
| OpenAI | GPT-4o, GPT-4 Turbo, GPT-3.5 |
| Anthropic | Claude 3.5 Opus, Sonnet, Haiku |
| Google (Gemini) | Gemini 1.5 Pro, Gemini 1.5 Flash |
| Meta (Llama) | Llama 3, Llama 3.1 |
| Mistral AI | Mixtral 8x22B, Mistral Large |
| Cohere | Command R+, Command R |
| AI21 Labs | Jurassic-2 |
| DeepSeek | DeepSeek-V2, DeepSeek-Coder |

## STACK LAYERS (OVERVIEW)

| | |
|---|---|
| 1. MODELS | Foundation models (LLMs) like GPT, Claude, Gemini, Llama |
| 2. ORCHESTRATION | Frameworks to connect models, tools and memory (LangChain, etc.) |
| 3. TOOLS & APIS | APIs and tools to enhance capabilities (Search, DB, Code, etc.) |
| 4. DATA & STORAGE | Vector DBs, databases and storage for knowledge |
| 5. DEPLOYMENT | Hosting, serving and monitoring (Cloud, Docker, Kubernetes) |
| 6. APPLICATION | UI / API / App (What user interacts with) |

## KEY COMPONENTS IN STACK

① **Models**
Choose the right LLM for your use case.

② **Orchestration Frameworks**
Manage prompts, chains, memory, agents.

③ **Tools & APIs**
Add external capabilities (search, code exec, DB access, file handling).

④ **Data Layer**
Store and retrieve knowledge efficiently.

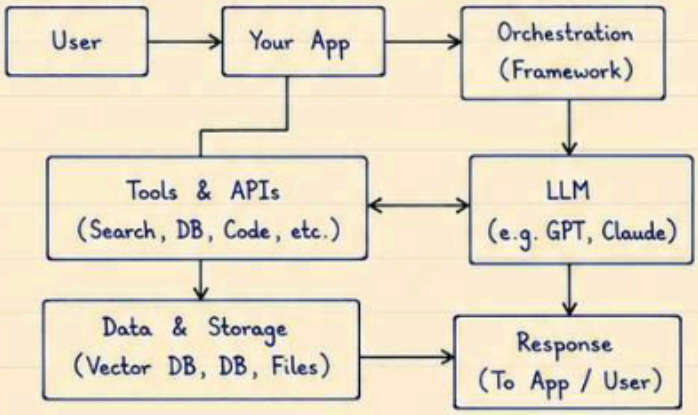
⑤ **Deployment Layer**
Deploy and scale your AI application.

⑥ **Application Layer**
Frontend / backend through which user interacts.

## POPULAR SERVICES & PLATFORMS

- OpenAI API (GPT Models)
- Anthropic API (Claude Models)
- Google Vertex AI (Gemini Models)
- Hugging Face (Models & Inference)
- Cohere API
- Mistral AI API
- Replicate (Run open models)
- Groq (Fast Inference)
- Pinecone (Vector Database)
- Supabase / Firebase (Backend as a Service)

## SIMPLE AI DEVELOPER STACK FLOW




TIP : Pick the right model + tools + data → Better AI Apps

Remember : Good Stack = Good Models + Good Tools + Good Data + Good Deployment

@Aman_Views

# 10. BUILD YOUR FIRST AI APP

## WHAT WILL WE BUILD?

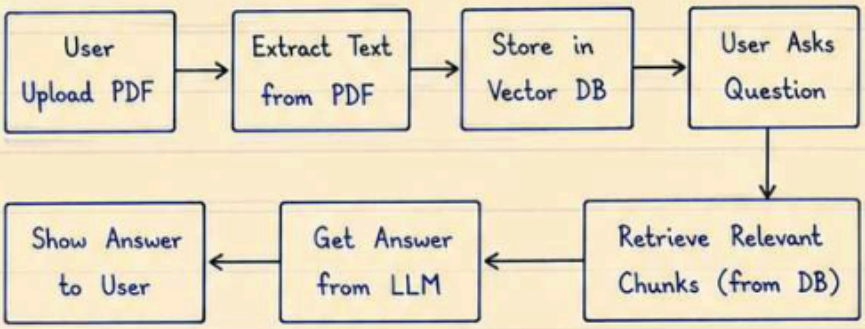
Let's build a simple AI App -

"Ask PDF"

Upload a PDF and ask questions.

AI will read the PDF and give answers.

## APP OVERVIEW (FLOW)




## TECH STACK WE WILL USE

- Frontend : Streamlit
- Language : Python
- LLM : OpenAI GPT-4o (or any model)
- Embeddings : OpenAI Embeddings
- Vector DB : FAISS (local)
- PDF Loader : PyPDF2 / LangChain Loader
- Framework : LangChain

## STEP-BY-STEP PLAN

1. User uploads a PDF file.
2. Extract text from the PDF.
3. Split text into small chunks.
4. Convert chunks into embeddings.
5. Store embeddings in Vector DB.
6. User asks a question.
7. Find relevant chunks from DB.
8. Send question + context to LLM.
9. Get answer and show to user.

## KEY Python LIBRARIES

| Library | Purpose |
|---|---|
| Streamlit | Build UI for the app |
| LangChain | Chains, LLMs, Document loaders |
| OpenAI | Access to LLM & Embeddings |
| FAISS | Vector similarity search |
| PyPDF2 | Read PDF files |
| tiktoken | Count tokens |

## PROJECT STRUCTURE (FOLDERS)

```
ai-pdf-app/
├── app.py              # Main Streamlit app
├── utils.py            # Helper functions
├── requirements.txt
├── data/               # Uploaded PDFs
└── vector_store/       # Saved FAISS index
```

## BASIC CODE FLOW (app.py)

1. Import libraries
2. Upload PDF
3. Process PDF (extract + split)
4. Create embeddings & store in FAISS
5. User question input
6. Retrieve context & call LLM
7. Show answer

## WHAT YOU WILL LEARN

- How to work with PDFs
- How embeddings & Vector DB work
- How to connect LLM with your data
- How to build a real AI application

### TIPS

- ★ Start small and keep it simple.
- ★ Understand each step before moving ahead.
- ★ Try with different PDFs.
- ★ Improve UI and add more features.

**Remember : Build → Experiment → Learn → Improve → Ship**